

Unidad 1: Concepto de NoSQL

Clase 1 – De lo relacional a NoSQL: nuevos paradigmas de datos

Objetivo

Comprender el surgimiento de las bases de datos NoSQL y analizar las diferencias fundamentales con el modelo relacional, identificando criterios de selección en función del volumen de datos, el tipo de información y los patrones de consulta.

Tema

- Presentación de la materia. Alcance, enfoque por competencias y TP integrador.
- Concepto de NoSQL.
- Modelo Relacional vs estructuras No Relacionales.
- Criterios de selección entre ambos modelos.
- Relación con volúmenes de datos y patrones de consulta.
- ACID en entornos NoSQL.

Introducción

En las últimas dos décadas hemos presenciado una **“tercera revolución” en la tecnología de bases de datos**, impulsada por las necesidades de aplicaciones modernas con alcance global y exigencia de alta disponibilidad continua. Este cambio dio origen al movimiento **NoSQL**, un término que significa *“Not Only SQL”* (no solo SQL), englobando una nueva generación de sistemas de gestión de datos no relacionales. En esta unidad se abordará el concepto de NoSQL, sus diferencias con el modelo relacional tradicional, criterios para elegir entre ambos paradigmas, la relación de dichas elecciones con los volúmenes de datos y patrones de consulta, y cómo se gestionan las propiedades ACID en entornos NoSQL. El objetivo es ofrecer una visión clara y técnica de estos conceptos, basada en bibliografía académica y documentación actual, con ejemplos reales de la industria para contextualizar las ideas.

Concepto de NoSQL: breve historia y evolución del paradigma

El surgimiento de NoSQL se remonta a la necesidad de manejar datos masivos y distribuidos que comenzaron a generar las aplicaciones web de escala global a partir de mediados de la década del 2000. Durante los **40 años previos**, las bases de datos relacionales (RDBMS - Relational Database Management System) dominaron casi sin rival gracias a sus sólidas bases teóricas (álgebra relacional de Codd, 1970), su modelo tabular intuitivo y el lenguaje SQL que facilitó las consultas a usuarios no programadores. Estos RDBMS ofrecían además rendimiento optimizado en hardware disponible y garantizaban la **consistencia absoluta de los datos** mediante

transacciones ACID en todo momento. Sin embargo, hacia finales de los 2000, la explosión de datos caracterizada por los “3 V” de **Big Data** –volumen, variedad y velocidad– evidenció las limitaciones del enfoque relacional tradicional.

En **2007–2009**, gigantes web como Google, Amazon y Facebook ya enfrentaban retos de escala inéditos: millones de usuarios concurrentes, datos semiestructurados de diversas formas (documentos JSON, actividad de redes sociales, registros de sensores, etc.) y la necesidad de desplegar servicios en múltiples centros de datos globales con alta disponibilidad. Tecnologías pioneras marcaron el camino: Google Bigtable (almacenamiento distribuido tipo columna ancha), el servicio Dynamo de Amazon (base NoSQL clave-valor altamente disponible) y proyectos open-source inspirados en ellos (Apache **Cassandra** en 2008, Apache **HBase** en 2008, **MongoDB** en 2009, Neo4j en 2009, entre otros). En 2009 se populariza el término “NoSQL” para agrupar a estos sistemas alternativos que **rompían con la arquitectura relacional+SQL+ACID tradicional**.

Vale aclarar que NoSQL no es una tecnología única, sino un **paraguas que abarca varias categorías de bases de datos no relacionales** surgidas en esta época. Todas ellas comparten el principio de **flexibilizar el modelo de datos y la consistencia** para lograr escalabilidad horizontal y manejar eficientemente grandes volúmenes. Las principales categorías incluyen:

- **Bases de datos clave-valor** – almacenan pares simples de clave y valor, típicamente usadas para *caching* y sesiones (ej. Redis, Amazon DynamoDB).
- **Bases de datos de documentos** – almacenan datos semiestructurados (generalmente JSON) permitiendo esquemas flexibles y anidados (ej. MongoDB, CouchDB).
- **Bases de datos de columnas anchas** – organizan datos en tablas escalar pero con filas de esquemas dinámicos y particionamiento eficiente por clave principal (ej. Cassandra, HBase), adecuadas para series de tiempo y grandes volúmenes distribuidos.
- **Bases de datos de grafos** – optimizadas para representar entidades y sus relaciones mediante nodos y aristas, facilitando consultas profundas de grafos (ej. Neo4j, Amazon Neptune).

BASES DE DATOS NoSQL



1 CLAVE -> VALOR

Organizan los datos en parejas de clave (un identificador) y valor (datos asociados al identificador).

Ej. En un carrito de compras, el identificador es el usuario, y el valor, los productos.



La BD NoSQL **priorizan la velocidad sobre la integridad de los datos** y permiten redundancias para agilizar las consultas. Se clasifican en:

2 DOCUMENTALES

Organizan los datos que se necesitan consumir en un documento JSON.

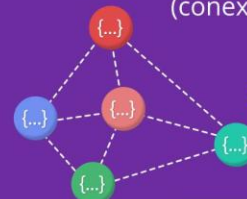
Ej. Los datos de un post de IG están en un solo archivo JSON.



3 GRAFOS

Tienen nodos (conjunto de datos como un JSON) y aristas (conexiones entre ellos).

Ej. Lo usan los e-commerce para recomendar productos.



En suma, la evolución histórica de NoSQL fue una respuesta “inevitable” a nuevos requisitos de la era Web 2.0 y Big Data. Como señala Harrison (2015), tras décadas de hegemonía del modelo relacional, **desde 2008 en adelante proliferó una explosión de sistemas no relacionales** que priorizan la *disponibilidad continua* y el *alcance global*. Esta tercera revolución en bases de datos demostró que “one size doesn’t fit all” (una sola solución no sirve para todos los casos), inaugurando la era de la **persistencia poliglota** donde se elige la base de datos según la naturaleza de los datos y las necesidades de la aplicación.

2. Modelo relacional vs. estructuras No Relacionales

El modelo relacional organiza la información en **tablas normalizadas**, con esquemas rígidos (columnas fijas por tabla) y relaciones explícitas mediante claves

primarias/foráneas. Este diseño garantiza integridad referencial y elimina redundancia, siendo excelente para **datos altamente estructurados** y transacciones complejas. Además, las RDBMS ofrecen un lenguaje estándar (SQL) capaz de realizar consultas ad hoc muy elaboradas (joins, agregaciones, subconsultas) sobre los datos estructurados. Un RDBMS tradicional siempre sigue las propiedades ACID estrictamente, de modo que las operaciones mantienen la base de datos en un estado consistente en todo momento. Esto da confianza en ámbitos como finanzas: por ejemplo, en una base relacional un débito bancario reducirá una cuenta **y** aumentará otra en la misma transacción o no hará ninguna de las dos, garantizando balances correctos.

Por el contrario, las bases de datos NoSQL abarcan **estructuras de datos más flexibles y diversas**. No imponen un esquema fijo: cada registro (documento, fila o nodo) puede tener un formato distinto o atributos variables. Esto las hace ideales para datos semiestructurados o en evolución. A continuación se resumen las **diferencias clave** entre ambos enfoques:

- **Modelo de datos:** Relacional utiliza un modelo **tabular** uniforme; NoSQL puede usar modelo **clave-valor, documentos JSON, columnas dinámicas o grafos**, según el tipo de base.
- **Esquema:** Relacional exige **esquemas fijos** y normalizados (p. ej., tablas separadas para entidades y relaciones); NoSQL permite **esquemas flexibles o inexistentes**, almacenando objetos complejos tal cual se usan en la aplicación (embebiendo sub-objetos o listas dentro de un documento, por ejemplo).
- **Integridad y consistencia:** Relacional proporciona **consistencia fuerte y restricciones ACID** a nivel global de la base de datos – tras cualquier transacción los datos cumplen todas las reglas y no existen actualizaciones parciales. NoSQL típicamente opta por un modelo **BASE** (*Basically Available, Soft state, Eventual consistency* – “básicamente disponible, estado suave, consistencia eventual”). Esto significa que garantizan alta disponibilidad de las operaciones y rendimiento, pero la consistencia puede ser *eventual*: diferentes nodos pueden tener valores desactualizados temporalmente y no se imponen todos los checks de integridad referencial automáticamente. **Nota:** Algunos sistemas NoSQL **sí** cumplen ACID en cierto grado (ver sección 5), pero en general la mayoría sacrifica algo de consistencia inmediata por escalabilidad.
- **Consultas y herramientas:** En SQL relacional, las consultas complejas son posibles mediante join de múltiples tablas, vistas, subconsultas, etc., con un **lenguaje declarativo estándar (SQL)** y herramientas analíticas maduras. En NoSQL, las consultas dependen del modelo: las bases de documentos suelen tener APIs de consulta JSON o lenguajes específicos (p. ej. MongoDB usa su propia sintaxis JSON o agregaciones; Cassandra ofrece CQL, similar a SQL pero limitado a un solo conjunto de columnas por consulta); las bases clave-valor normalmente recuperan datos solo por llave primaria; las de grafos usan lenguajes como Cypher o Gremlin para navegar relaciones. En general, **NoSQL prioriza la simplicidad y rapidez de operaciones típicas (lecturas/escrituras directas)** por sobre la flexibilidad de consultas ad hoc multientidad. De hecho, muchas bases NoSQL **no soportan uniones complejas** ni subconsultas; la

responsabilidad de combinar datos recae en la lógica de la aplicación o en modelos de datos desnormalizados.

- **Desnormalización vs. normalización:** El enfoque relacional tiende a normalizar datos en múltiples tablas relacionadas, minimizando duplicación. Las NoSQL a menudo **desnormalizan** datos – es decir, replican ciertos campos en distintos documentos o filas – para evitar costosos joins y hacer lecturas más eficientes. Por ejemplo, en una base de documentos, un solo documento de *Orden de Compra* podría incluir embebida la información completa del cliente y los productos, en lugar de referencias a tablas separadas como haría una base SQL. Esto reduce el número de accesos necesarios para armar la información completa, a costa de duplicación de datos.
- **Escalabilidad:** Este punto, vital en la práctica, separa fuertemente ambos modelos. Las bases **relacionales escalan verticalmente** en general: para manejar más carga, se mejora el hardware del servidor (CPU, RAM, disco más rápidos) o se recurre a replicación para balancear principalmente cargas de lectura. El escalado horizontal (añadir más servidores) es complicado en RDBMS, pues mantener sincronizada una **base de datos distribuida con ACID fuerte** requiere mecanismos complejos (clústers con sharding manual, federación, dos fases de commit, etc.). Por el contrario, las bases **NoSQL escalan horizontalmente de forma más natural**, ya que fueron diseñadas para particionar datos en múltiples nodos sin interdependencias fuertes. Es relativamente sencillo distribuir la carga en decenas o centenas de servidores, ya que no hay joins ni transacciones multi-nodo que coordinar en muchos casos. Esta diferencia se refleja en el rendimiento: una base relacional puede verse limitada por el hardware de un solo nodo para ciertas operaciones intensivas, mientras que una base NoSQL puede repartir esas operaciones entre nodos logrando throughput mucho mayor agregadamente.

En resumen, **las bases relacionales brindan estructura estricta, consultas poderosas y consistencia total**, siendo ideales para datos altamente estructurados con relaciones complejas (ej. sistemas financieros, ERP, etc.). **Las NoSQL ofrecen flexibilidad de datos y alta escalabilidad**, adecuadas para datos semiestructurados o muy voluminosos donde se privilegia rendimiento y disponibilidad sobre las garantías de consistencia inmediata. No se trata de que un modelo sea “mejor” que el otro en términos absolutos, sino de entender las compensaciones: *cada paradigma favorece ciertos casos de uso*. A continuación profundizamos en cómo decidir entre uno u otro.

3. Criterios de selección: ¿Cuándo elegir modelo relacional vs. NoSQL?

Escoger entre una base relacional y una NoSQL depende de múltiples factores relativos a la aplicación y sus requisitos. A nivel universitario es importante dominar estos criterios de diseño, pues una correcta selección de tecnología de datos impacta en la escalabilidad, mantenibilidad y éxito de un proyecto. **Principales criterios a considerar:**

- **Estructura y tipo de datos:** Si los datos son **altamente estructurados y homogéneos**, con un esquema estable (por ejemplo, registros de transacciones bancarias, donde cada registro tiene los mismos campos bien definidos), un

RDBMS relacional es apropiado. En cambio, si se manejarán **datos semiestructurados o sin esquema fijo** – por ejemplo, documentos JSON con campos flexibles, datos anidados, o diferentes esquemas por elemento – es más eficiente usar una base NoSQL de documentos o clave-valor. Un caso típico: aplicaciones de gestión de contenido o catálogos de productos donde cada ítem puede tener atributos distintos (tallas, colores, especificaciones técnicas variables). En una base relacional esto requeriría tablas muy amplias con muchos campos nulos u tablas dinámicas difíciles de manejar; en una base de documentos (MongoDB, Couchbase) cada documento de producto puede almacenar solo los atributos relevantes sin penalizaciones.

- Relaciones entre datos:** Si el dominio del problema involucra **muchas relaciones cruzadas o JOINS complejos** (por ejemplo, un sistema académico con estudiantes, cursos, inscripciones, calificaciones, donde se necesitan consultar relaciones *muchos a muchos* frecuentemente), las bases relacionales sobresalen. Su diseño normalizado permite mantener integridad referencial y consultar eficientemente mediante JOIN. Por otro lado, hay situaciones donde las relaciones son importantes pero **no encajan bien en tablas relacionales**: un ejemplo clásico es una red social de usuarios y amigos. En SQL, modelar la amistad exige una tabla de relaciones (ej. tabla Amistades con columnas usuario_id y amigo_id), donde un usuario con cientos de amigos tendría cientos de filas separadas. Navegar la red de amistades vía SQL (amigos de amigos, intereses comunes, etc.) resulta costoso. En cambio, una base de datos de grafos puede almacenar ese mismo usuario como un nodo con una lista de conexiones a sus amigos en **un solo registro**, optimizado para recorrer relaciones directamente. Por ello, **para datos altamente conectados** (redes sociales, gráficos de conocimiento, detección de fraudes mediante relaciones entre transacciones, etc.) es recomendable un NoSQL de grafos como Neo4j. En casos de relaciones simples y jerarquías estrictas, una base documental también puede bastar embebiendo subdocumentos (por ejemplo, almacenar un pedido con una lista de sus ítems dentro del mismo documento). En suma: *si las relaciones y su integridad son centrales, RDBMS; si las relaciones son complejas (grafo denso) o pueden manejarse embebidas sin join (jerarquías), considerar NoSQL.*
- Escalabilidad y volumen de datos esperado:** Este es frecuentemente el punto decisivo. Para **volúmenes de datos modestos o crecimientos previsibles**, un RDBMS puede funcionar bien y simplificar la consistencia. Pero si se anticipa **crecimiento masivo de datos (orden de terabytes a petabytes) o picos altísimos de transacciones**, las bases NoSQL suelen manejar mejor la escala horizontal. Por ejemplo, aplicaciones globales con millones de usuarios activos (redes sociales, juegos *online*, plataformas de streaming) han tenido éxito con NoSQL porque pueden agregar nodos al clúster conforme crece la demanda sin re-arquitectura profunda. Un sistema relacional tradicional enfrentando esa escala requeriría costosos servidores de alta gama y aún así sería un reto mantener el desempeño. En general, **cuando la tasa de escrituras o lecturas por segundo se cuenta en decenas de miles o más, o la base de datos excede las capacidades de un solo nodo**, es señal para optar por NoSQL. Casos reales: Apple informó operar más de 75.000 nodos Cassandra almacenando más de 10

petabytes de datos, con clústers individuales de más de 1000 nodos manejando millones de operaciones por segundo – algo impráctico de lograr con una base relacional centralizada. En el sector retail, Best Buy migró ciertas funciones a Cassandra para absorber picos de tráfico estacionales (como **7x** el tráfico normal en ventas navideñas), logrando atender >50.000 solicitudes por segundo de forma “*impecable*”. Estos ejemplos ilustran que a escalas masivas, una arquitectura distribuida NoSQL es más conveniente para mantener el rendimiento y disponibilidad.

- **Patrones de consulta y acceso:** Hay que analizar cómo se accederán los datos. Si la aplicación requiere **consultas transaccionales complejas, con múltiples agregaciones, filtrados ad hoc y flexibilidad en los reportes**, las bases relacionales ofrecen la potencia de SQL y generalmente optimizadores de consulta sofisticados. En cambio, muchas aplicaciones modernas siguen patrones de acceso más simples: por ejemplo, *workloads* tipo *OLTP* que realizan siempre lecturas/escrituras por clave primaria (get/put de un ID), sin necesidad de combinaciones complejas. Si las consultas están bien definidas de antemano (conocidos como **patrones de acceso predecibles**), se puede diseñar un modelo NoSQL optimizado específicamente para esas consultas. Un ejemplo es una aplicación de IoT donde cada sensor envía lecturas de temperatura con timestamp; típicamente se accede a las lecturas por rango de tiempo por sensor. Un almacén de tipo columnas anchas (Cassandra) permite particionar las filas por sensor y ordenar por timestamp, logrando consultas de series temporales muy eficientes, mucho más que en una tabla SQL genérica con todos los sensores mezclados. **En resumen:** para *ad-hoc queries* y flexibilidad analítica, mejor un modelo relacional; para *acceso predefinido* (p. ej. siempre por clave o por rangos conocidos) en grandes volúmenes, un modelo NoSQL bien diseñado será más eficiente.
- **Consistencia y requisitos de transacciones:** Si la aplicación no puede tolerar *ninguna* anomalía de consistencia (p. ej. en sistemas financieros, contables, de inventario crítico, etc.), y requiere transacciones multi-objeto frecuentes, el modelo relacional ACID sigue siendo el estándar de oro. En situaciones donde una **ligera latencia en la sincronización de datos es aceptable** (por ejemplo, en redes sociales es tolerable que un *post* nuevo tarde unos segundos en aparecer para todos los usuarios globalmente), entonces un modelo eventualmente consistente puede ser viable. Muchas aplicaciones web priorizan disponibilidad: prefieren seguir funcionando aunque algunos nodos estén desactualizados, antes que bloquearse esperando sincronización. Por lo tanto, **si la carga de trabajo es principalmente operaciones independientes por registro (sin transacciones multi-entidad)** y se valora más la disponibilidad/performance que la consistencia inmediata, un NoSQL es candidato fuerte. En cambio, si se necesitan **transacciones complejas, agregaciones globales con precisión** (ej. generar balances financieros correctos al centavo al instante), la robustez de un SQL ACID es difícil de igualar. Un término medio son los sistemas *NewSQL* o algunas NoSQL modernas que implementan transacciones distribuidas, pero suelen conllevar mayor complejidad.
- **Madurez del ecosistema y soporte:** Las RDBMS existen hace ~50 años, con gran cantidad de herramientas de administración, reportes, BI, personal

capacitado (DBAs), etc. En NoSQL, aunque hay madurez creciente, cada tecnología tiene su propia interfaz y menos estándares. Si el equipo de desarrollo está más familiarizado con SQL y la curva de aprendizaje de una NoSQL específica es un problema, podría influir la decisión. NoSQL a menudo requiere incorporar lógica de manejo de consistencia en la aplicación, lo que aumenta complejidad. En entornos empresariales con necesidad de auditoría, reportes complejos o cumplimiento (ej. SQL ofrece ACID que facilita consistencia para auditorías), se suele optar por no abandonar totalmente los SQL tradicionales, o combinar ambos enfoques (persistencia poliglota: usar cada tipo de base de datos para lo que mejor hace).

En la práctica, **muchas soluciones emplean una mezcla**: por ejemplo, mantener un **core** transaccional en un RDBMS (p. ej. la información contable crítica) y usar NoSQL para módulos de la aplicación que requieren alta escalabilidad o flexibilidad (p. ej. sesiones de usuario, registros de actividad, caché de consultas frecuentes, etc.). Este enfoque híbrido permite beneficiarse de lo mejor de cada mundo. Guy Harrison (2015) denomina esto *polyglot persistence*, reconociendo que aplicaciones modernas suelen usar múltiples motores de datos especializados.

En conclusión, la selección relacional vs. NoSQL debe basarse en un **análisis caso por caso**. Se recomienda evaluar: estructura de datos (fijo vs variable), volumen y tasa de crecimiento, tipo de operaciones (transaccionales complejas vs accesos simples), necesidad de consistencia estricta vs disponibilidad, y experiencia tecnológica del equipo. Tomar la decisión adecuada garantiza que la base de datos soporte eficientemente las necesidades presentes y futuras de la aplicación.

4. Volumen de datos y patrones de consulta: impacto en la elección

Como extensión de lo anterior, es útil profundizar en cómo **el volumen de datos y los patrones de consulta** influyen en la arquitectura de datos.

Volúmenes masivos (Big Data): Cuando hablamos de datos de magnitud de terabytes a petabytes, especialmente si crecen rápidamente, las arquitecturas distribuidas NoSQL muestran su fortaleza. Por diseño, muchas bases NoSQL se ejecutan en clústeres de decenas o cientos de servidores comunes (*commodity hardware*), donde cada nodo almacena una partición del total de datos. Así, el volumen total se reparte y es manejable en conjunto. Por ejemplo, Apache Cassandra fue diseñado siguiendo el modelo de Dynamo (Amazon) para escalar casi linealmente al agregar nodos: empresas como **Apple** han operado decenas de miles de nodos Cassandra para servicios en la nube (como iCloud), almacenando decenas de petabytes. Este nivel de volumen distribuido sería muy difícil de lograr con una única base de datos SQL centralizada, tanto por límites físicos (espacio en disco, memoria) como por la sobrecarga de coordinar transacciones y bloqueos en un solo motor.

Además, NoSQL maneja bien los **datos de muy alta velocidad (streaming)**. Por ejemplo, en sistemas de registro de eventos (logs) o métricas en tiempo real, donde ingresan miles de eventos por segundo, una base de datos de columnas anchas como

Cassandra o HBase puede absorber escrituras secuenciales muy rápido gracias a su modelo de *log estructurado* (LSM trees) y replicación asíncrona, sin incurrir en bloqueos transaccionales globales. Esto ha hecho que industrias como telecomunicaciones e IoT adopten NoSQL para recopilar flujos de datos sensor a gran escala. Un caso ilustrativo es **CERN** (organismo de investigación nuclear) que utiliza Cassandra para almacenar series de tiempo generadas por miles de dispositivos y aplicaciones de monitoreo, cumpliendo requerimientos de tasas de ingreso de datos muy altas con flexibilidad y escalabilidad.

En cambio, para volúmenes de datos **pequeños o moderados**, y especialmente si caben en un solo servidor potente, las bases relacionales pueden rendir excelentemente y brindar todas sus ventajas. De hecho, para un dataset de, digamos, pocos gigabytes y algunos cientos de consultas por segundo, un RDBMS bien afinado puede manejarlo sin problemas en hardware moderno. Usar NoSQL en un escenario de bajo volumen podría añadir complejidad innecesaria (por ejemplo, carecer de joins nativos, tener que programar la lógica de consistencia manualmente) sin aprovechar realmente su escalabilidad. Por eso, *la escala prevista* es crucial: **NoSQL se justifica verdaderamente a gran escala** o en presencia de alta distribución geográfica de datos.

Patrones de consulta (workloads): Si la carga de trabajo consiste en muchas **lecturas aleatorias complejas**, como buscar datos por distintos campos no índices, realizar agregaciones arbitrarias o reportes globales, las bases relacionales (o almacenes analíticos columnares SQL) suelen ser más apropiados. Un Data Warehouse relacional, por ejemplo, está optimizado para consultas agregadas globales que recorren gran parte de los datos. En cambio, muchas bases NoSQL sobresalen en **patrones de acceso conocidos y simples**: por ejemplo, *lookup* directo por clave, o consultas basadas en un campo particionado. Un patrón típico es el de **consulta por clave primaria**, que NoSQL resuelve en tiempo constante ($O(1)$) al ir directamente al nodo y partición donde reside la clave. Otro patrón es **scan por rango** (por ejemplo, todas las entradas de cierta fecha a otra para una entidad), que bases de columnas como Cassandra pueden manejar eficientemente si la clave de partición y clustering está diseñada para ese rango.

Por otro lado, NoSQL suele ser la elección para **altas tasas de escritura** en patrones sencillos. Sistemas de métricas, logging, tracking de clics, etc., generan mucho más escrituras que lecturas muchas veces. Ahí, una base de datos eventual consistente que **no incurre en locking por cada escritura** puede alcanzar throughput muy superior. Por ejemplo, **BlackBerry** encontró necesario pasar de Postgres a Cassandra para su plataforma IoT, debido a la necesidad de ingestar volúmenes enormes de eventos de dispositivos en tiempo real; Cassandra pudo escalar en escritura donde la base relacional original no llegaba. En general, **workloads orientados a escritura** (como registros de actividad, telemetría) son un punto fuerte de NoSQL.

Un patrón de consulta particular que merece mención es el de **agregados geográficos o demográficos en tiempo real** (propios de analítica en dashboards, etc.). Si se requiere contar/agrupar datos a medida que llegan (ej. cuántos usuarios activos por región en los últimos 5 minutos), a veces conviene usar stores NoSQL especializados

(p.ej. base de series de tiempo, o incluso Redis para contadores en memoria) que sacrifiquen consistencia estricta pero brinden respuestas rápidas actualizadas eventual o aproximadamente. Alternativamente, implementar eso en un SQL podría requerir consultas intensivas a tablas enormes, impactando rendimiento.

Disponibilidad geográfica: Cuando los usuarios y datos están distribuidos globalmente, mantener una única base de datos en un centro de datos puede introducir latencia de red significativa. Muchas NoSQL ofrecen replicación geográfica fácil, sirviendo datos desde nodos localizados cercanos a los usuarios y tolerando fallos de zona. Por ejemplo, **Apache Cassandra** y **MongoDB** permiten replicar datos en múltiples centros de datos y dirigir las consultas al más cercano, reduciendo latencia. Lograr esto con una base SQL tradicional es más complejo (aunque existen soluciones como Galera Cluster para MySQL o la base distribuida CockroachDB, que intentan dar SQL con replicación global, conocidas como NewSQL). Si la aplicación requiere presencia multi-región activa, suele ser más sencillo con NoSQL eventualmente consistente, ya que no necesita un *lock* global – en caso de partición de red entre regiones, cada una puede seguir operando y sincronizar después (esto deriva del teorema CAP, ver sección 5). Por tanto, los patrones de acceso global y la tolerancia a particiones apuntan hacia NoSQL.

Resumen: Volúmenes muy grandes + patrones de acceso predecibles o simples = terreno ideal para NoSQL (clave-valor, documental, columna, según el caso). Volúmenes acotados + necesidad de flexibilidad de consulta = terreno para SQL relacional. No obstante, siempre es posible que un sistema requiera *ambos* tipos de acceso: por eso se ven arquitecturas mixtas, donde por ejemplo se almacena la misma información en un NoSQL optimizado para ciertos accesos rápidos, y en paralelo se carga en una base SQL para reporting complejo (*ELT pipelines*, etc.). Un ejemplo real: muchas compañías de comercio electrónico usan **Redis (clave-valor en memoria)** para almacenamiento temporal de carritos de compra o perfiles en sesión (rápido y escalable para millones de usuarios concurrentes), pero luego consolidan los datos finales de órdenes en un SQL central para contabilidad y BI. Este tipo de pipeline combina las fortalezas de cada enfoque.

5. ACID en entornos NoSQL

Las propiedades **ACID** (Atomicidad, Consistencia, Aislamiento, Durabilidad) son la base de las transacciones confiables en bases de datos tradicionales. En entornos **NoSQL**, **originalmente la mayoría de sistemas optó por relajar las garantías ACID** en pos de rendimiento y particionado distribuido. Esto suele resumirse diciendo que los sistemas NoSQL siguieron el modelo **BASE** (Basic Availability, Soft state, Eventual consistency) en lugar de ACID. A continuación, se analiza cómo se manejan (o no) las propiedades ACID en distintos sistemas NoSQL:

- **Atomicidad:** En bases relacionales, la atomicidad asegura que una serie de operaciones en una transacción ocurren *todas o ninguna*. En NoSQL, típicamente **la atomicidad se limita a nivel de registro individual** o clave-partición. Por ejemplo, en Cassandra cualquier inserción o actualización de las

columnas dentro de una misma partición (fila) es atómica a ese nivel. Es decir, si se escriben varias columnas en una fila, o se borra una fila entera, la operación se aplica completa o no se aplica. Sin embargo, las bases NoSQL *no* garantizan atomicidad **multientidad**: no es posible por defecto agrupar actualizaciones a dos documentos distintos o dos filas de distinta partición en una única transacción atómica. Si falla una parte, las demás pueden haber quedado aplicadas (no hay *rollback* automático global). Algunos NoSQL ofrecen “**transacciones ligeras**” o chequeos de condición que brindan algo de atomicidad multi-fila, pero usualmente con costo en rendimiento (ej.: *lightweight transactions* de Cassandra para compare-and-set, usando protocolo Paxos).

- **Consistencia:** Aquí es crucial distinguir entre *consistencia referencial ACID* (todas las reglas de integridad se cumplen después de cada transacción) y *consistencia de replicación* en sistemas distribuidos (que está relacionada al teorema CAP). Las bases NoSQL clásicas optan por **consistencia eventual**: tras una actualización, no todos los nodos ven inmediatamente el nuevo valor, pero **eventualmente** (en segundos, ms o minutos según el sistema) todos convergen. Esto significa que durante breves ventanas de tiempo, un cliente puede leer datos “viejos” mientras otro ya ve el nuevo, sin que eso viole las garantías del sistema (es aceptable por diseño). Por ejemplo, en una base clave-valor replicada, si se actualiza la clave X en el nodo A, otro cliente que consulte la réplica B milisegundos después podría obtener el valor anterior si la actualización aún no se propagó. Esta inconsistencia temporal es tolerada a cambio de mayor disponibilidad. En contraste, un sistema relacional replicado suele configurarse en modo **consistencia fuerte** (p. ej., replicación síncrona), donde una transacción no se considera completa hasta que todas las réplicas estén actualizadas – lo que aumenta la latencia o reduce disponibilidad si alguna réplica falla. Muchas NoSQL permiten ajustar un **nivel de consistencia tunable**: por ejemplo, Cassandra tiene niveles de lectura/escritura QUORUM, ALL, ONE, etc., donde uno puede elegir exigir respuestas de cierta cantidad de nodos para considerar válida una operación. Un nivel QUORUM (mayoría) mejora la consistencia leído, pero puede sacrificar algo de disponibilidad.
Consistencia referencial: las NoSQL en general **no aplican automáticamente restricciones de integridad referencial ni unicidad más allá de la clave primaria**. Depende de la aplicación mantener, por ejemplo, que si se borra un objeto se limpien sus referencias en otros documentos, etc. Algunas bases NoSQL proveen ciertas facilidades (como esquemas JSON con validaciones en MongoDB, o transacciones multi-documento recientes, ver más abajo), pero la mayoría de los sistemas originales no lo hacían. En Redis, por ejemplo, se garantiza consistencia interna de los datos (no se corromperá la estructura de un dato en medio de una operación), pero **no hay integridad relacional** como en SQL – mantener la coherencia lógica entre distintos valores es responsabilidad del desarrollador.
- **Aislamiento:** El aislamiento ACID asegura que transacciones concurrentes no interfieran, logrando un efecto como si se ejecutaran en secuencia. En NoSQL, dado que por lo general no hay transacciones complejas, el aislamiento suele darse de forma más limitada. Muchos sistemas sacrifican aislamiento estricto

para rendimiento: por ejemplo, las escrituras en un clúster distribuido pueden aplicarse en orden diferente en distintos nodos, permitiendo *anomalías* *fenómenos* como lecturas sucias si no se configura cuidadosamente. Dicho esto, varios NoSQL ofrecen **aislamiento a nivel de registro**: mientras se escribe un documento o fila, otras operaciones no ven un estado intermedio. Cassandra garantiza aislamiento a nivel de fila (una actualización a una fila no es visible parcialmente). MongoDB, antes de tener transacciones, aseguraba aislamiento al nivel de documento: una operación que modifica un documento es atómica y aislada, otros hilos no lo ven a medio modificar. Redis, al ser monohilo, naturalmente ejecuta comandos uno por uno, logrando un aislamiento **serializado** para cada comando. Si se usan *transacciones MULTI* en Redis, se alinean los comandos y se ejecutan juntos, aislados de otros clientes durante ese momento. En general, **los NoSQL implementan aislamiento al mínimo nivel práctico** (documento/fila), pero no aíslan conjuntos arbitrarios de operaciones a múltiples objetos sin soporte transaccional explícito.

- **Durabilidad:** Este aspecto varía según el sistema. Muchos NoSQL sí persisten los datos de forma confiable en disco, pero algunos (especialmente los orientados a memoria como Redis) **priorizan velocidad sobre durabilidad por defecto**. Por ejemplo, Redis almacena datos en RAM para rapidez, y la durabilidad es opcional mediante *snapshots* periódicos en disco o un log Append Only (AOF). Si no se configura persistencia, un apagón del servidor puede implicar pérdida de los últimos datos en memoria (no es *durable ACID*). En cambio, bases como Cassandra son durables: cada escritura se registra en un **commit log en disco** además de en memoria, de modo que ante una caída se pueden recuperar las operaciones no aplicadas. MongoDB también garantiza durabilidad grabando en su journal las operaciones antes de confirmarlas. Neo4j (grafo) es otro ejemplo de NoSQL durable y ACID completo: sus transacciones se registran y confirman a disco. **El compromiso típico en NoSQL fue sacrificar algo de durabilidad síncrona** (p. ej., permitir confirmar una operación tras escribir en memoria y replicar a N nodos, incluso si no todos lo guardaron en disco aún) para lograr menor latencia. Muchos sistemas usan la técnica de *escritura anticipada en log* que luego se descarga a almacenamiento persistente en segundo plano. Esto implica que en ciertos escenarios (fallo catastrófico múltiple) podría perderse la última fracción de datos comprometidos. Sin embargo, correctamente configurados, la mayoría de NoSQL de producción aseguran durabilidad suficiente para no perder datos confirmados – aunque a veces el usuario debe elegir modos (por ej., en MongoDB la opción `writeConcern majority` asegura que el dato está en varios nodos antes de confirmar).

En los primeros años de NoSQL, el enfoque general fue **“AP sobre CA”** en términos de CAP: sistemas que *Prefieren disponibilidad y partición* toleradas sobre la consistencia estricta inmediata. Esto llevó al lema *“Eventual consistent, highly available”*. Un resultado interesante es que algunas garantías ACID se retomaron con el tiempo a medida que los motores NoSQL maduraron. Por ejemplo:

- **MongoDB:** Inicialmente ofrecía solo atomicidad por documento. Pero desde *MongoDB 4.0 (2018)* incorporó **transacciones ACID multi-documento** en un solo clúster, y desde la versión 4.2 (2019) soporta transacciones distribuidas incluso en clústeres *sharded*. Esto fue un gran avance para cubrir casos de uso que antes requerían SQL. Hoy MongoDB es ACID-compliant a nivel de transacciones multi-documento, haciendo que en ciertos escenarios actúe similar a un SQL tradicional en cuanto a seguridad de datos. Este movimiento mostró la demanda de desarrolladores por combinar la flexibilidad NoSQL con garantías transaccionales cuando son necesarias.
- **Neo4j:** Desde su origen, la base de datos de grafos Neo4j decidió ser totalmente **ACID**. Al ser orientada a grafos, su nicho incluye aplicaciones como gestión de redes sociales, recomendaciones o fraudes financieros, donde la consistencia de las relaciones es crucial. Neo4j garantiza transacciones atómicas y aisladas sobre el grafo, asegurando consistencia y durabilidad de cada operación. Esto la pone en la categoría de NoSQL *CP* (consistente y tolerante a particiones, pero sacrificando disponibilidad en caso de falla en cluster). Muchos la consideran prácticamente una base de datos *orientada a grafos pero con cualidades de una base relacional* en términos de ACID.
- **Cassandra:** Sigue un modelo *AP* (alta disponibilidad, consistencia ajustable). No implementa ACID global: no hay concepto de rollback de transacciones fallidas ni integridad referencial automática. Sin embargo, proporciona **consistencia ajustable** y **“atomicidad” en rango limitado**. Ofrece también operaciones de inserción idempotentes y soporte de *Lightweight Transactions* para casos que requieren secuencialidad (internamente usa Paxos para operaciones estilo compare-and-set, logrando aislamiento serializable para esas operaciones específicas). Cassandra garantiza *Atomicidad a nivel partición, Aislamiento a nivel fila, Durabilidad por replicación y commit log; Consistencia eventual tunable*. Esto significa que, por ejemplo, **una transferencia bancaria no es trivial de asegurar en Cassandra** – habría que implementar la lógica de compensación manualmente o usar sus transacciones ligeras con cuidado, dado que mover dinero de A a B no es automáticamente atómico en múltiples filas. Por ello, Cassandra y similares se aplican en casos donde tales multi-entidad transacciones no son la norma (o se manejan a nivel de aplicación).
- **Redis:** Es un caso especial por ser en memoria. Redis ofrece **atomicidad y aislamiento** de forma natural por su ejecución monohilo: cada comando se completa íntegro antes de atender el siguiente, así que nunca dos operaciones se intercalan a medias. También tiene bloques de MULTI/EXEC para ejecutar varias operaciones de forma seguida, que asegura que todas se procesen secuencialmente una detrás de otra sin intromisión. En ese sentido cumple A, C (consistencia interna) e I. No obstante, **la Durabilidad en Redis es opcional**: si no se configura AOF o snapshot, los datos se perderán en un crash. Por eso decimos que Redis tiene *“ACID parcial: A-C-I fuertes, Durabilidad débil”*. En entornos que usan Redis, a menudo se acepta esta limitación porque se emplea para datos temporales (cache, sesiones, rankings) donde perderlos no es crítico o se puede regenerar. Para usar Redis en algo crítico, se recomienda activar AOF (persistencia en disco) y replicación. De lo contrario, **no debería usarse solo para, por ejemplo, registros financieros permanentes**.

En la formulación teórica, Eric Brewer planteó el **Teorema CAP** que dice que en sistemas distribuidos no se pueden garantizar simultáneamente Consistencia fuerte, Alta Disponibilidad y Tolerancia a Particiones; se debe sacrificar al menos uno. Los sistemas relacionales tradicionales en un solo servidor evitan la partición (por eso CAP no aplicaba estrictamente, operaban en CA, no toleran particiones: ante partición simplemente no hay sistema distribuido). Al distribuir datos, las NoSQL eligieron típicamente ser **AP (alta disponibilidad incluso con particiones, pero sin consistencia fuerte)**. Esto se alinea con BASE: garantizar respuesta siempre (aunque sea con datos potencialmente viejos) y resiliencia a caídas de nodos, a cambio de relajar la sincronía de la consistencia. Otros sistemas optan por **CP (consistencia fuerte y tolerancia a fallos de red, sacrificando disponibilidad en casos extremos)** – es el caso de muchas bases de grafos o de los NewSQL (por ejemplo, Google Spanner, CockroachDB, etc., que prefieren bloquear o retrasar respuestas antes que servir datos inconsistentes). No existe un modelo perfecto universal: **ACID vs BASE es una decisión de diseño según las necesidades del dominio**. En sectores como banca, salud o gobierno, la confiabilidad de ACID es obligatoria, mientras que en aplicaciones web de gran escala, BASE suele ser aceptable y deseable para rendimiento.

Vale destacar que la línea entre SQL y NoSQL en cuanto a ACID se ha ido difuminando. Hoy en día algunos motores NoSQL ofrecen **transacciones ACID completas** (MarkLogic, Neo4j, MongoDB, etc.), y también han surgido bases de datos relacionales nuevas con arquitectura distribuida para escalabilidad (Por ejemplo, Google Spanner o CockroachDB combinan particionado automático con transacciones ACID, posicionándose como “NewSQL”). Esto evidencia que la industria busca integrar lo mejor de ambos mundos cuando es posible. No obstante, implementar ACID en entornos distribuidos conlleva complejidad y coste en rendimiento, por lo que siempre habrá lugar para sistemas eventualmente consistentes más simples y veloces en ciertos casos de uso.

En síntesis, los entornos NoSQL originalmente optaron por no proveer transacciones ACID globales, apoyándose en el desarrollador para manejar la consistencia. Esto permitió escalar y distribuir datos masivamente. Algunas propiedades ACID (como durabilidad y atomicidad local) sí se mantienen en muchos NoSQL, pero otras (consistencia global, aislamiento estricto) se relajan. Con la evolución, han aparecido excepciones de NoSQL ACID-compliant, demostrando que ACID y escalabilidad no son mutuamente excluyentes en todos los casos – aunque conseguir ambas suele implicar compromisos (latencia mayor, algoritmos de consenso, etc.). En cualquier caso, es fundamental que el ingeniero de datos comprenda las garantías que ofrece (o no) la base de datos seleccionada. Al diseñar una aplicación, se deben considerar preguntas como: *¿Puedo aceptar lecturas desactualizadas? ¿Necesito transacciones multiobjeto? ¿Qué pasa ante una caída de nodo?* Las respuestas guiarán si se necesita un sistema ACID tradicional, un NoSQL BASE, o algo intermedio.

Conclusiones

El paradigma NoSQL ha enriquecido el ecosistema de gestión de datos al proveer alternativas al modelo relacional para escenarios donde este último muestra rigidez o

dificultades de escala. Tras revisar su concepto, historia, diferencias con las bases relacionales, criterios de selección, implicaciones de volumen/patrón de uso y manejo de ACID, podemos concluir:

- **NoSQL no sustituye completamente a SQL**, sino que lo complementa. Surgió por necesidades específicas (Big Data, datos semiestructurados, alta concurrencia distribuida) y ha demostrado su valía en esos campos, mientras que los RDBMS siguen siendo óptimos en casos de estructura fija, transacciones complejas y analítica flexible. En entornos reales es común emplear ambos tipos según el componente del sistema (por ejemplo, una empresa puede usar SQL para su contabilidad interna y NoSQL para su módulo web de sesiones de usuario).
- **Entender las ventajas y trade-offs** de cada modelo es crucial para la ingeniería de datos. Un diseño adecuado de arquitectura de datos puede implicar grandes ahorros de costos y mejoras de rendimiento. Por ejemplo, elegir un almacén clave-valor para caché puede reducir la carga de la base principal relacional; o migrar un módulo de logs de auditorio a una base distribuida de columnas puede permitir retener más datos históricos a menor costo.
- Los criterios de selección deben siempre alinearse con los *requerimientos no funcionales*: escalabilidad requerida, tolerancia a fallos, consistencia esperada, latencia permisible, complejidad de desarrollo, etc. No existe una “mejor base de datos” en abstracto, sino una más adecuada para cada caso de uso.
- **ACID vs BASE**: se debe decidir cuánta inconsistencia temporal es aceptable. Muchas aplicaciones web pueden vivir con consistencia eventual (mostrando quizás datos ligeramente retrasados) a cambio de no caer ante alto tráfico. Otras aplicaciones no lo tolerarían. Hoy en día, incluso si se adopta NoSQL, hay opciones de implementar ciertas garantías ACID cuando se necesitan (como las transacciones de MongoDB o el uso de mecanismos de confirmación en Cassandra). La clave está en conocer bien las herramientas y configurar correctamente los niveles de consistencia y replicación.

Para cerrar, la ingeniería de datos moderna promueve una **visión poliglota**: utilizar la herramienta de almacenamiento de datos que mejor se adapte a cada componente. Un ingeniero de datos debe estar familiarizado tanto con las bases de datos relacionales tradicionales como con los diferentes tipos de NoSQL (documentos, grafos, etc.), sabiendo sus fortalezas, debilidades y casos de uso típicos. Solo con esa perspectiva amplia podrá diseñar sistemas de datos robustos, escalables y coherentes con las necesidades de la era del *Big Data*. En clases posteriores profundizaremos en cada categoría de NoSQL y sus casos de estudio, cimentando la base conceptual presentada en este informe.

Bibliografía

- Harrison, Guy (2015). *Next Generation Databases: NoSQL, NewSQL, and Big Data*. Apress. (Capítulos 1 y 3: historia evolutiva de las bases de datos y surgimiento de NoSQL).

- Pivert, Olivier (2018). *NoSQL Data Models: Trends and Challenges*. Wiley-ISTE. (Sección 1.1: introducción a NoSQL, Big Data, teorema CAP y modelo BASE vs ACID).
- Deka, Ganesh Chandra (2017). *NoSQL: Database for Storage and Retrieval of Data in Cloud*. Chapman & Hall/CRC. (Texto de referencia sobre categorías de NoSQL en entornos de nube; capítulos introductorios sobre comparativa con modelo relacional y transacciones).
- Bradberry, Russell; Lubow, Eric (2013). *Practical Cassandra: A Developer's Approach*. Addison-Wesley. (Guía práctica de Cassandra; consulta de casos de uso y fundamentos de escalabilidad en bases de datos de columnas anchas).
- **Documentación oficial MongoDB – MongoDB Multi-Document ACID Transactions**, MongoDB Inc. (Whitepaper, 2018) y Manual de referencia (sección Transactions). Incluye el anuncio de soporte transaccional multi-documento desde versión 4.0 y mejores prácticas.
- **Documentación Apache Cassandra – DataStax Cassandra OSS Documentation** (versión 2.2, 2022): “How are Cassandra transactions different from RDBMS transactions?” y secciones de Consistency/Availability. Explica las garantías A, I, D en Cassandra y el modelo de consistencia eventual tunable.
- **AWS Database Blog (2023) – Comparativa: bases de datos relacionales vs. no relacionales**. Artículo en español de Amazon Web Services que resume las diferencias de modelo, rendimiento, escalado y casos de uso para RDBMS vs NoSQL. Incluye ejemplos como el modelado de amigos en redes sociales en NoSQL.
- **Apache Cassandra Case Studies (2023)** – Página oficial del proyecto Cassandra con casos de uso reales en industria. Referencia de ejemplos citados: Apple (petabytes de datos, 75k nodos), Best Buy (picos de 50k req/s), CERN (monitorización con Cassandra), entre otros.
- **Neo4j Developer Blog (2023) – Data Consistency Models: ACID vs. BASE Explained**, Neo4j.com. Discusión sobre en qué consisten ACID y BASE, sus trade-offs, y mención de que Neo4j y ciertos NoSQL son ACID-compliant.
- **Medium Blog – Bit Magic (2025). “Partial ACID: How Redis handles Atomicity, Consistency & Isolation”**. Explicación detallada de las propiedades ACID en Redis, destacando su enfoque en ACI y la durabilidad opcional, con recomendaciones de uso seguro.
- Vaughan, Jack (2018). **“MongoDB 4.0 takes ACID transactions to multi-document level.”** *TechTarget – SearchDataManagement*. (Noticia del lanzamiento de MongoDB 4.0, útil para contextualizar la importancia de ACID en NoSQL y el enfoque de MongoDB para combinar modelos).